

Virtual File System (VFS)

Project Documentation

Project Overview

Technology Used

- **Programming Language:** C/C++
- **Libraries:** Standard C libraries (`stdio.h`, `stdlib.h`, `string.h`, `unistd.h`), C++ I/O (`iostream`)
- **Operating System Concepts:** File systems, inodes, superblocks, file descriptors, system calls

User Interface Used

- **Command-Line Interface (CLI):** The project uses a text-based interface where users input commands to interact with the virtual file system (e.g., `create`, `read`, `write`, `ls`, etc.).

Platform Required

- **Operating System:** Linux/Unix-like systems (due to the use of `unistd.h` and system call implementations)
- **Compiler:** GCC or any C/C++ compiler supporting C++ standards

Hardware Requirements

- **Minimal Hardware:** Any system with a CPU, RAM, and storage capable of running a C/C++ compiler
- **Recommended:** A modern computer with at least 2GB RAM and a multi-core processor for efficient compilation and execution

Description of the Project

This project implements a **Virtual File System (VFS)** that simulates basic file system operations in memory, mimicking the behavior of a real file system. It supports operations like file creation, reading, writing, deleting, and metadata management using a simplified structure. The VFS operates entirely in memory, without interacting with physical storage, making it a lightweight simulation for educational purposes.

Data Structures Used

1. **Superblock (SUPERBLOCK):** Stores metadata about the file system, such as total and free inodes.
2. **Inode (INODE):** Represents a file's metadata, including name, size, type, permissions, and data buffer.
3. **File Table (FILETABLE):** Manages open file details like read/write offsets, mode, and inode pointer.

4. **User File Descriptor Table (UFDT)**: Maps file descriptors to file table entries.

5. **Linked List**: Used to link inodes for easy traversal.

Diagram of Data Structures

Superblock

```
+-----+
| TotalInodes  |
| FreeInode    |
+-----+
```

Inode (Linked List)

```
+-----+ +-----+ +-----+
| FileName   |----->| FileName   |----->| FileName   |
| InodeNumber |         | InodeNumber |         | InodeNumber |
| FileSize    |         | FileSize    |         | FileSize    |
| FileActualSize|       | FileActualSize|       | FileActualSize|
| FileType    |         | FileType    |         | FileType    |
| Buffer       |         | Buffer       |         | Buffer       |
| LinkCount    |        | LinkCount    |        | LinkCount    |
| ReferenceCount|       | ReferenceCount|       | ReferenceCount|
| Permission   |        | Permission   |        | Permission   |
| Next-----|         | Next-----|         | Next-----|
+-----+ +-----+ +-----+
```

File Table

```
+-----+
| readoffset  |
| writeoffset  |
| count       |
| mode        |
| ptrinode----|--> Inode
+-----+
```

UFDT Array

```
+-----+
| ptrfiletable--|--> File Table
+-----+
| ptrfiletable--|--> File Table
+-----+
| ...          |
+-----+
```

The Flow of the Project

1. **Initialization:** The program initializes the superblock (`InitialiseSuperBlock`) and creates a Disk Inode List Block (`CreateDILB`) with 50 inodes.
2. **Command Loop:** The program enters an infinite loop, accepting user commands via `fgets`.
3. **Command Parsing:** Commands are parsed using `sscanf` into up to four parts (e.g., `create File_name Permission`).
4. **Command Execution:** Based on the command, functions like `CreateFile`, `OpenFile`, `ReadFile`, etc., are called.
5. **Error Handling:** Each operation checks for errors (e.g., invalid parameters, permissions) and displays appropriate messages.
6. **Termination:** The loop exits when the user enters `exit`.

File System Concepts

What is a File System?

A **file system** is a method used by operating systems to manage and organize files on storage devices. It defines how data is stored, retrieved, and managed, including file naming, permissions, and metadata.

File Systems Used by Linux and Windows

- **Linux:**
 - **ext4:** Widely used, supports large file systems, journaling.
 - **XFS:** High-performance, suitable for large files.
 - **Btrfs:** Advanced features like snapshots, compression.
- **Windows:**
 - **NTFS:** Default for modern Windows, supports large files, permissions, encryption.
 - **FAT32:** Older, widely compatible but limited file size.
 - **exFAT:** Optimized for flash drives, supports large files.

Parts of a File System

1. **Superblock:** Stores metadata about the file system (e.g., total inodes, free space).
2. **Inodes:** Metadata for individual files (e.g., size, permissions, data location).
3. **Data Blocks:** Store the actual file content.
4. **Directory Structure:** Organizes files in a hierarchical structure.
5. **File Allocation Table:** Tracks file locations (used in some file systems like FAT).

UAREA and Its Contents

The **UAREA** (User Area) is a per-process data structure in Unix-like systems that stores process-specific information. In the context of this VFS, it is not explicitly implemented but can be related to the `UFDT` array, which maintains file descriptors for open files. Typical UAREA contents include:

- Process ID
- File descriptor table
- Current working directory
- User credentials (UID, GID)
- Signal handlers

File Table and Its Contents

The **File Table** (`FILETABLE`) tracks open file information for a process:

- **readoffset**: Current position for reading.
- **writeoffset**: Current position for writing.
- **count**: Number of references to the file table entry.
- **mode**: Access mode (READ, WRITE, or READ+WRITE).
- **ptrinode**: Pointer to the associated inode.

Use: It manages the state of open files, allowing the system to track read/write positions and permissions.

InCore Inode Table and Its Use

The **InCore Inode Table** is a memory-resident copy of inode metadata for files currently in use. In this project, it is represented by the `head` linked list of `INODE` structures. **Use**:

- Provides quick access to file metadata without disk I/O.
- Tracks file status (e.g., open files, reference counts).
- Supports operations like reading, writing, and seeking.

What is an Inode?

An **inode** (index node) is a data structure that stores metadata about a file (not the file name or data). It uniquely identifies a file in the file system.

Contents of Superblock

The superblock (`SUPERBLOCK`) contains:

- **TotalInodes**: Total number of inodes (50 in this project).
- **Freelnode**: Number of free inodes available.

Types of Files

1. **Regular Files**: Contain user data (e.g., text, binary).

2. **Directory Files:** Store file names and inode mappings.
3. **Special Files:** Device files (e.g., block, character devices).
4. **Symbolic Links:** Pointers to other files.
5. **Pipes/Sockets:** For inter-process communication.

Contents of Inode

The `INODE` structure contains:

- **FileName:** Name of the file (up to 50 characters).
- **InodeNumber:** Unique identifier for the inode.
- **FileSize:** Maximum size (2048 bytes).
- **FileActualSize:** Current size of data.
- **FileType:** Type (REGULAR or SPECIAL).
- **Buffer:** Pointer to file data in memory.
- **LinkCount:** Number of hard links.
- **ReferenceCount:** Number of open references.
- **permission:** Access mode (1=READ, 2=WRITE, 3=READ+WRITE).
- **next:** Pointer to the next inode.

Use of a Directory File

A **directory file** stores mappings of file names to inode numbers, enabling hierarchical organization and file lookup. In this project, directories are not explicitly implemented, but the inode linked list simulates a flat directory.

Operating System File Security

Operating systems maintain file security through:

- **Permissions:** Read, write, execute permissions for owner, group, and others.
- **Ownership:** User ID (UID) and Group ID (GID) associated with files.
- **Access Control Lists (ACLs):** Fine-grained access control (not implemented in this project).
- **Encryption:** Protects file content (not implemented here). In this VFS, security is simplified to permission checks (READ, WRITE, READ+WRITE).

What Happens When a User Opens a File?

1. The `OpenFile` function checks if the file exists (`Get_Inode`).
2. Verifies if the requested mode (READ, WRITE, or READ+WRITE) is allowed.
3. Allocates a file table entry in `UFDTArr`.
4. Initializes read/write offsets and increments the inode's reference count.
5. Returns a file descriptor (index in `UFDTArr`).

What Happens When a User Calls `lseek`?

The `LseekFile` function:

1. Validates the file descriptor and seek origin (`START`, `CURRENT`, `END`).
2. For **READ** or **READ+WRITE** mode:
 - Adjusts `readoffset` based on the origin and size.
 - Ensures the new offset is within bounds.
3. For **WRITE** mode:
 - Adjusts `writeoffset` and updates `FileActualSize` if necessary.
4. Returns 0 on success or -1 on error.

Difference Between Library Function and System Call

- **Library Function:** A higher-level function provided by a library (e.g., `stdio.h`), executed in user space, often wrapping system calls for convenience (e.g., `printf`).
- **System Call:** A low-level interface to the OS kernel, executed in kernel space, providing direct access to OS services (e.g., `open`, `read`).

Use of This Project

- **Educational Tool:** Teaches file system concepts like inodes, superblocks, and file operations.
- **Simulation:** Simulates a file system in memory for learning without affecting physical storage.
- **Prototype:** Demonstrates file system operations for academic or experimental purposes.

Difficulties Faced

1. **Memory Management:** Ensuring proper allocation and deallocation of `Buffer` and `FILETABLE` to avoid leaks.
2. **Error Handling:** Managing various error cases (e.g., file not found, permission denied).
3. **Command Parsing:** Handling variable-length commands with `sscanf`.
4. **Linked List Traversal:** Efficiently managing the inode linked list for operations like `Get_Inode`.

Improvements Needed

1. **Directory Support:** Add hierarchical directory structures.
2. **Persistence:** Save the file system state to disk.
3. **Advanced Permissions:** Implement user/group-based permissions.
4. **Concurrency:** Support multiple users/processes with thread safety.
5. **Extended Commands:** Add more Unix-like commands (e.g., `cp`, `mv`).

System Calls: Internal Working

open

- **Function:** `OpenFile(char *name, int mode)`

- **Working:**

1. Validates input (non-null name, valid mode).
2. Checks if the file exists using `Get_Inode`.
3. Verifies if the requested mode is allowed by the file's permissions.
4. Allocates a `FILETABLE` entry in `UFDTArr`.
5. Sets `readoffset/writeoffset` based on mode and increments `ReferenceCount`.
6. Returns the file descriptor or an error code.

close

- **Function:** `CloseFileByName(char *name)` or `CloseFileByName(int fd)`

- **Working:**

1. Retrieves the file descriptor using `GetFDFromName` (for name-based close).
2. Resets `readoffset` and `writeoffset` to 0.
3. Decrements the inode's `ReferenceCount`.
4. Returns 0 on success or -1 if the file is not found.

read

- **Function:** `ReadFile(int fd, char *arr, int isize)`

- **Working:**

1. Validates the file descriptor and checks if the file is open.
2. Ensures the mode and permissions allow reading.
3. Checks if `readoffset` is at the end of the file.
4. Copies data from the inode's `Buffer` to the provided array (`arr`).
5. Updates `readoffset` and returns the number of bytes read.

write

- **Function:** `WriteFile(int fd, char *arr, int isize)`

- **Working:**

1. Validates the file descriptor, mode, and permissions for writing.
2. Checks if there's enough space (`MAXFILESIZE`).
3. Copies data from `arr` to the inode's `Buffer` at `writeoffset`.
4. Updates `writeoffset` and `FileActualSize`.
5. Returns the number of bytes written.

lseek

- **Function:** `LseekFile(int fd, int size, int from)`

- **Working:**

1. Validates the file descriptor and seek origin.
2. For reading: Adjusts `readoffset` based on `START`, `CURRENT`, or `END`.

3. For writing: Adjusts `wrioffset` and updates `FileActualSize` if needed.
4. Ensures offsets are within valid bounds.
5. Returns 0 on success or -1 on error.

stat

- **Function:** `stat_file(char *name)`
- **Working:**
 1. Searches for the file using `Get_Inode`.
 2. Displays metadata (name, inode number, size, permissions, etc.).
 3. Returns 0 on success or -1/-2 for invalid parameters or file not found.

chmod

- **Not Implemented:** The project does not support changing file permissions dynamically. Permissions are set during file creation (`CreateFile`).

unlink

- **Function:** `rm_File(char *name)`
- **Working:**
 1. Retrieves the file descriptor using `GetFDFromName`.
 2. Decrements the inode's `LinkCount`.
 3. If `LinkCount` reaches 0, sets `FileType` to 0 and frees the file table.
 4. Increments `FreeInode` in the superblock.
 5. Returns 0 on success or -1 if the file is not found.

Command Explanations

ls

- **Use:** Lists all files in the directory.
- **VFS Implementation:** `ls_file()` displays file names, inode numbers, sizes, and link counts for non-zero `FileType` inodes.

ls -l

- **Use:** Lists files with detailed information (permissions, size, etc.).
- **VFS Equivalent:** Similar to `ls_file()`, but could be extended to show permissions like `stat_file`.

ls -a

- **Use:** Lists all files, including hidden ones (starting with `.`).

- **VFS Limitation:** No hidden files in this VFS, so `ls` is equivalent.

rm

- **Use:** Deletes a file.
- **VFS Implementation:** `rm_File(name)` reduces `LinkCount` and frees resources if `LinkCount` is 0.

cat

- **Use:** Displays file contents.
- **VFS Equivalent:** `ReadFile` can be used via the `read` command to output file contents.

cd

- **Use:** Changes the current directory.
- **VFS Limitation:** No directory support; not implemented.

chmod

- **Use:** Changes file permissions.
- **VFS Limitation:** Not implemented; permissions are set during creation.

cp

- **Use:** Copies files.
- **VFS Limitation:** Not implemented; would require creating a new inode with copied data.

df

- **Use:** Displays disk space usage.
- **VFS Equivalent:** Could display `FreeInode` and `TotalInodes` from `SUPERBLOCKobj`.

find

- **Use:** Searches for files.
- **VFS Equivalent:** `Get_Inode` can search by name but is not a full `find` command.

grep

- **Use:** Searches file contents for patterns.
- **VFS Limitation:** Not implemented; would require parsing file buffers.

ln

- **Use:** Creates hard or symbolic links.

- **VFS Limitation:** Hard links are partially supported via `LinkCount`, but no explicit `ln` command.

mkdir

- **Use:** Creates directories.
- **VFS Limitation:** No directory support; not implemented.

pwd

- **Use:** Prints the current working directory.
- **VFS Limitation:** No directory support; not implemented.

touch

- **Use:** Creates empty files or updates timestamps.
- **VFS Equivalent:** `CreateFile` creates a new file with no data.

uname

- **Use:** Displays system information.
- **VFS Limitation:** Not implemented; irrelevant to this in-memory VFS.

stat

- **Use:** Displays file metadata.
- **VFS Implementation:** `stat_file(name)` and `fstat_file(fd)` display inode details.

man

- **Use:** Displays command documentation.
- **VFS Implementation:** `man(name)` provides usage and description for supported commands.

mkfs

- **Use:** Formats a file system.
- **VFS Equivalent:** `InitialiseSuperBlock` and `CreateDILB` initialize the VFS.